Pack 01 · Speed physics

Why decode is limited by memory, prefill by math, and how batching changes both. Lessons 01–04, 06, 09, 15, 16, 24, 25, 28, 44, 59–61. Answer each question out loud in about a minute, then check the card.

Q1 · "Can we advertise 300 tokens/s?" PRIORITY

Lessons: 02, 06, 16, 24

> **INTERVIEWER**
>
> We serve Llama 3 8B in BF16 on one H100 with a single user online. The PM wants to advertise 300 tokens/s streaming, and a teammate's benchmark already shows 2,000 tokens/s. Can we promise it?

60-second answer

- **No.** With one user (batch 1), every decode step has to read **all the weights** from HBM once, so a step can never be faster than weight bytes ÷ memory bandwidth.
- The 8B model in BF16 is 16 GB, so each step takes at least 16 GB ÷ 3.35 TB/s ≈ **4.8 ms per token**. Each step produces 1 token, so the ceiling is 1 ÷ 4.8 ms ≈ **200 tokens/s**, and real kernels land around 150 to 180. That's why we can't promise 300.
- That means the 2,000 tokens/s benchmark is broken. The usual bugs are **not calling GPU synchronize** before stopping the timer, which matters because kernels run asynchronously and the timer stops before the work finishes, and **counting prompt tokens as output**.
- To go faster per user, there are two levers: read fewer bytes per step (quantization), or get more tokens out of each weight read (speculative decoding).

Key numbers: decode floor = weight bytes ÷ bandwidth = $16 \times 10^9 \div 3.35 \times 10^{12} \approx 4.8$ ms.

Likely follow-up: "Is our 70B FP8 baseline healthy at 14 ms TPOT on two H100s with tensor parallelism?" With tensor parallelism the weights are split across the two GPUs, and each reads its half at the same time, so the bandwidths add up to 6.7 TB/s. The 70 GB of weights then take 70 GB ÷ 6.7 TB/s ≈ 10.4 ms, which is the floor. MBU is floor ÷ measured, so 10.4 ÷ 14 ≈ 74%, which is healthy.

> **TRAP**
>
> I once said the ceiling comes from TFLOPS. Decode is limited by memory bandwidth. With tensor parallelism, add the GPUs' bandwidths.

Q2 · Why is a 500-token prefill only about 2× one decode step? PRIORITY

Lessons: 04, 15, 16, 44

> **INTERVIEWER**
>
> Prefill of a 500-token prompt takes about 8 ms; one decode token takes about 4.8 ms. Why so close, and how long do 30 streaming users freeze when someone pastes an 8,000-token document?

60-second answer

- Each phase is limited by a different resource, and the time is whichever limit is slower: time ≈ **max(FLOPs ÷ peak FLOPS, bytes ÷ bandwidth)**.
- **Decode** produces one token per step, so it does very little math for each byte it reads. That means it's limited by reading the weights, which takes about **4.8 ms**.
- **Prefill** processes all 500 prompt tokens in parallel while reading the weights only once, so the math dominates. The pass costs $2 \times 8 \times 10^9 \times 500 \approx 8 \times 10^{12}$ FLOP, and $8 \times 10^{12} \div 989$ TFLOPS ≈ **8 ms**. That's why it's only about 2× one decode step.
- An 8,000-token paste costs $2 \times 8 \times 10^9 \times 8{,}000 \approx 1.3 \times 10^{14}$ FLOP, which takes $1.3 \times 10^{14} \div 989$ TFLOPS ≈ **130 ms**. Every other stream waits for it, and since 130 ÷ 4.8 ≈ 27, each user freezes for about **27 decode steps**. Chunked prefill fixes this (pack 03).

Key numbers: forward-pass FLOPs ≈ 2 × parameters × tokens; H100 dense BF16 ≈ 989 TFLOPS.

Likely follow-up: "What's the ridge point?" It's the number of FLOPs per byte where the two limits meet: 989 TFLOPS ÷ 3.35 TB/s ≈ 295 FLOP per byte. If the work does fewer FLOPs per byte than that, you're memory-bound, and if it does more, you're compute-bound.

> **TRAP**
>
> I divided bytes by FLOPS. That isn't a time. Bytes go over bandwidth, FLOPs over FLOPS, and prefill cost must scale with the prompt length.

Q3 · "32 users means 32× slower, right?" PRIORITY

Lessons: 02, 16, 25, 28

> **INTERVIEWER**
>
> A PM says serving 32 users at once makes each user 32× slower. Llama 3 8B in BF16 on an H100; each user holds about 1,000 tokens of KV at 128 KiB per token. What really happens to per-user speed and total throughput?

60-second answer

- **One user alone:** each decode step reads all 16 GB of weights, which takes 16 GB ÷ 3.35 TB/s ≈ **4.8 ms**. Each step produces 1 token, so the user gets 1 ÷ 4.8 ms ≈ **200 tokens/s**.
- **32 users batched:** the **16 GB of weights is still read once per step, shared by all 32 users**. The extra cost is each user's own KV cache: 32 × 1,000 tokens × 128 KB ≈ 4.1 GB. The step reads 16 + 4.1 ≈ 20.1 GB, so it takes 20.1 GB ÷ 3.35 TB/s ≈ **6.0 ms**, and that one step produces **32 tokens**, one for each user.

- **What each user feels:** they still get 1 token per step, but a step now takes 6.0 ms instead of 4.8 ms. Since 6.0 ÷ 4.8 = 1.25, each user is **25% slower**: about 167 tokens/s instead of 200.
- **What the GPU produces:** 32 tokens every 6 ms, so 32 ÷ 0.006 s ≈ **5,300 tokens/s**. Compared with 200 tokens/s alone, that's 5,300 ÷ 200 ≈ **25×**.
- It's 25× and not the full 32× because the step got 1.25× longer: 32 ÷ 1.25 ≈ 25.6.
- The price of batching is **KV memory**, which grows with batch size × context length.

Key numbers: step ≈ (weights + batch × context × KV per token) ÷ bandwidth; total tokens/s = batch ÷ step; per user = 1 ÷ step.

Likely follow-up: "Our FastAPI server calls `generate()` and 20 users wait 40 s, with the GPU's math mostly idle. Bigger GPU?" No: requests run one at a time, so the queue is the problem. Continuous batching lets all 20 share each weight read.

> **TRAP**
> I added one user's KV instead of 32 users'. If the question says 32 users, a 32 must appear in the byte sum. Check ms → tokens/s against batch 1 (4.8 ms ≈ 200).

Q4 · When does decode become compute-bound? SOLID

Lessons: 02, 04, 61

> **INTERVIEWER**
> With short contexts, you keep raising the decode batch. At what batch does decode flip from memory-bound to compute-bound, and why doesn't that happen as early in practice?

60-second answer

- For each token, BF16 decode does 2 FLOP per parameter and reads 2 bytes per parameter. The weight bytes are **shared across the batch**, so a batch of B does 2 × B FLOP for the same 2 bytes, which means the intensity ≈ **batch size** FLOP per byte.
- Decode turns compute-bound once that intensity passes the H100 ridge of ≈ 295 FLOP per byte, so at a batch of **about 300**.
- In practice it happens much later because of the **KV cache**. The KV read grows with batch × context, and unlike the weights, attention's KV reads are never shared between users. So with long contexts, those extra bytes keep decode memory-bound much longer.

Key numbers: ridge = peak FLOPS ÷ bandwidth ≈ 989 ÷ 3.35 ≈ 295.

Likely follow-up: "So what does batching buy once you're compute-bound?" Nothing more per weight read. The math is already the bottleneck, so total throughput flattens while each user's latency keeps rising. That's the saturation wall in a concurrency sweep.

> **TRAP**
> say what the ratio is (FLOP per byte) and that the weight bytes are shared by the batch while KV bytes are not.

Q5 · "GPU-util is 100%, we need more GPUs" PRIORITY

Lessons: 09, 16, 59, 60

> **INTERVIEWER**
> One user chats with Llama 3 8B on an H100. The dashboard shows GPU-util 100%, power 350 W of a 700 W cap, and 6 ms per decode token. The manager wants more GPUs. Is it maxed out?

60-second answer

- **GPU-util** only tells you that some kernel was running during each sampling window, which is a duty cycle. It says nothing about how much of the chip that kernel is using, so 100% doesn't mean maxed out.
- The better measure is **MBU**. Each 6 ms step reads the 16 GB of weights, so the achieved bandwidth is 16 GB ÷ 6 ms ≈ 2.7 TB/s, and 2.7 ÷ 3.35 ≈ **80%** of peak bandwidth. Compute (MFU), meanwhile, is nearly idle.
- That's why **half power is expected**: decode is memory-bound, so the tensor cores spend most of their time waiting for data.
- So the fix is **more users per GPU** (batching), not more GPUs.

Key numbers: MBU = achieved bandwidth ÷ peak bandwidth = floor ÷ measured step.

Likely follow-up: "What would you put on the dashboard instead?" I'd show MBU and MFU, so you can see how busy memory and compute really are. Next to those I'd put tokens/s, KV cache usage, the number of running and waiting requests, and TTFT and TPOT percentiles.

> **TRAP**
> I read half power as "half the units idle, so the kernels need work", and divided by the ridge instead of peak bandwidth.

Q6 · What does a million tokens cost? PRIORITY

Lessons: 01, 02

> **INTERVIEWER**
> An H100 rents for $3 an hour. What do a million output tokens cost at one user (~200 tokens/s) and at 64 users batched (~4,300 tokens/s)?

60-second answer

- The GPU is billed by the hour, so first turn speed into tokens per hour: tokens/s × 3,600.
- **One user:** 200 × 3,600 = 720,000 tokens per hour, so a million tokens takes 1,000,000 ÷ 720,000 ≈ 1.4 hours. At $3 an hour that's ≈ **$4.17 per million**.
- **64 users batched:** 4,300 × 3,600 ≈ 15.5 M tokens per hour, so a million tokens costs $3 ÷ 15.5 ≈ **$0.19**. That's about 21× cheaper.
- The GPU-hour costs the same either way, and only the number of tokens you get out of it changes. That's why batching is the main cost lever.

Key numbers: cost per 1 M = $/hour ÷ (tokens/s × 3,600) × 10^6.

Likely follow-up: "Where does the 64-user step time come from?" Each step reads the 16 GB of weights once, plus every user's KV cache: 64 users × 4,096 tokens × 128 KB ≈ 34 GB. Together that's about 50 GB, so the step takes 50 GB ÷ 3.35 TB/s ≈ 15 ms. Each user gets 1 token per step, so 1 ÷ 15 ms ≈ 67 tokens/s per user, and 64 × 67 ≈ 4,300 tokens/s in total.

> **TRAP**
> I divided dollars by hours instead of multiplying, and slipped a power of ten (10^6 ÷ 200 is 5,000 s, not 500).

Q7 · Kernel launches and Amdahl PRIORITY

Lessons: 03, 04, 09, 60

> **INTERVIEWER**
> A decode step launches about 300 kernels at about 4 μs of CPU time each, against an ideal step of 4.8 ms. How much does that cost, and how do you cut it? Separately, a teammate made attention (30% of the step) 3× faster and claims decode is now 3× faster. What's the real speedup?

60-second answer

- **Launch overhead:** 300 kernels × 4 μs each = 1.2 ms of CPU time per step. Against a 4.8 ms step, 1.2 ÷ 4.8 = **25%**, so a quarter of the step goes to launching kernels.
- The fixes are **CUDA graphs**, where you record the step once and replay it with a single launch, and **kernel fusion**, which leaves fewer kernels to launch.
- **Amdahl:** attention is 30% of the step, so the other 70% doesn't change. The new time is 0.7 + 0.3 ÷ 3 = 0.8 of the old one, so the speedup is 1 ÷ 0.8 = **1.25×**, not 3×.
- Even if attention were free, the 70% would still be there, so the ceiling is 1 ÷ 0.7 ≈ **1.43×**. The lesson is to speed up the part that dominates the time.

Key numbers: overall speedup = 1 ÷ ((1 − p) + p ÷ s).

Likely follow-up: "Two optimizations: attention 30% made 3×, MLP 50% made 2×?" Work in times, not speedups. The untouched part is 100 − 30 − 50 = 20%, attention shrinks to 0.3 ÷ 3 = 0.1, and the MLP

shrinks to $0.5 \div 2 = 0.25$. The new time is $0.2 + 0.1 + 0.25 = 0.55$, so the speedup is $1 \div 0.55 \approx 1.8\times$. Speedups never add or multiply; you add the new times.

> **TRAP**
>
> I offered "fewer bytes" as a fix for launch overhead (that speeds up the bandwidth part), and gave the ceiling as "70%" instead of a ratio of times.

Build it yourself

- `exercises/03_04_amdahl.py`: an Amdahl calculator for several non-overlapping optimizations. It drills time accounting instead of adding speedups. Run `uv run exercises/03_04_amdahl.py`.